Regresión con máquinas de vectores de soporte

Sesión 6 · Módulo 2: Modelos de regresión

Margen blando · Tubo ε-insensible · Truco del kernel · GridSearchCV

Consumo energético de edificios · scikit-learn

Plan de la sesión (≈45 min)

Qué veremos

- **A.** Intuición de SVM de margen duro y de margen blando
- **B.** La pérdida ε-insensible y los vectores de soporte
- **C.** Truco del kernel: núcleos lineal, RBF y polinomial
- **D.** Ajuste de C, ε y parámetros del kernel con `GridSearchCV`

Una sola idea de fondo: en vez de penalizar **todos** los errores (como OLS o Ridge), SVR solo penaliza los que se salen de una banda de tolerancia alrededor de la predicción.

A1 · De dónde venimos: otra forma de controlar la complejidad

Lo que vimos en la Sesión 5

- Ridge y Lasso controlan el sobreajuste **penalizando el tamaño de los pesos** (α·penalización).
- Siguen siendo **modelos lineales**: una recta o un hiperplano.
- Útiles cuando la relación real es, más o menos, lineal.

¿Y si la relación no es lineal?

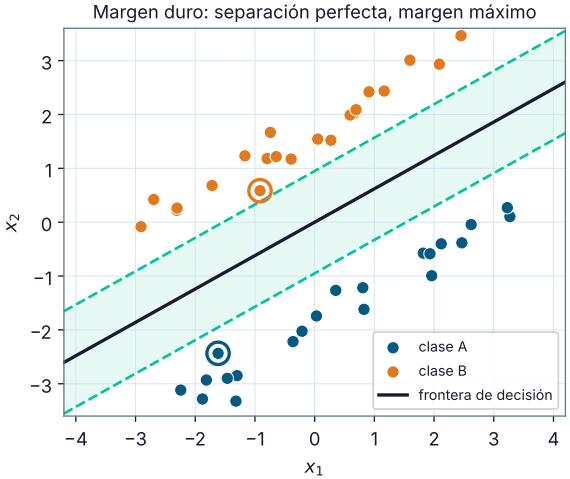
El consumo eléctrico de un edificio no sube ni baja siempre igual con la temperatura: sube con **frío** y con **calor**. Hoy vemos un modelo que puede curvarse.

De dónde viene SVR

- Nace de las **máquinas de vectores de soporte (SVM)**, creadas para **clasificar**.
- La misma idea de "margen" se adapta para **predecir un número**: así nace SVR.
- Para entender SVR conviene repasar primero el margen en clasificación.

Hoy: margen (SVM) $\rightarrow$ tubo de tolerancia (SVR) $\rightarrow$ truco del kernel $\rightarrow$ ajuste de hiperparámetros.

A2 · Margen duro: separar con el margen más amplio posible



La idea

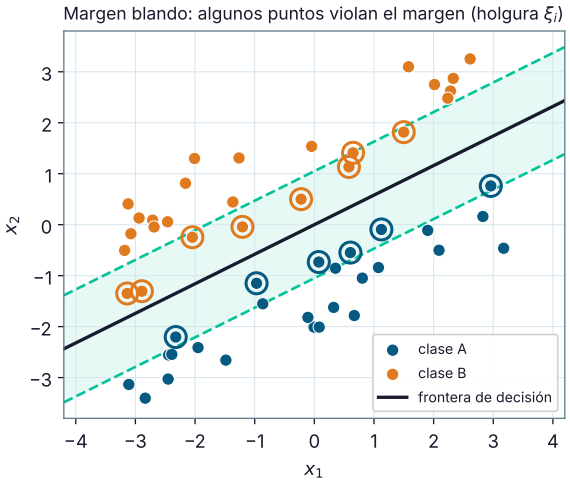
- De todas las fronteras que separan las clases, SVM elige la que deja el **margen más ancho** posible a ambos lados.
- Más margen → más **confianza** en puntos nuevos cercanos a la frontera.

Vectores de soporte

Solo los puntos **más cercanos** a la frontera (marcados con un círculo) determinan dónde queda el margen. Mover cualquier otro punto **no cambia nada**.

"Duro" porque exige separación perfecta: ningún punto puede quedar del lado equivocado.

A3 · Margen blando: tolerar violaciones con el parámetro C



El problema del margen duro

- Los datos reales casi nunca son perfectamente separables; un solo punto ruidoso puede **romper** el margen duro.
- El margen **blando** permite que algunos puntos violen el margen, a cambio de una **penalización** (holgura ξ_i).

El parámetro C

C controla qué tanto se penaliza violar el margen: C **grande** $\rightarrow$ pocas violaciones toleradas (puede sobreajustar); C **chico** $\rightarrow$ margen más ancho, más tolerante (puede subajustar).

A3b · Formulación matemática: el problema de optimización de SVM

Problema primal (margen blando)

$$\min_{w,\,b,\,\xi} \quad \tfrac{1}{2}\|w\|^2 \;+\; C\sum_{i=1}^{n}\xi_i$$

$$\text{sujeto a} \quad y_i\big(w^\top x_i + b\big) \geq 1 - \xi_i, \qquad \xi_i \geq 0, \qquad i = 1, \ldots, n$$

Cómo leerlo

- $\tfrac{1}{2}\|w\|^2$: minimizarlo equivale a **maximizar el ancho del margen** (el ancho de la franja en A2 es $2/\|w\|$).
- $\xi_i \geq 0$: cuánto viola el punto i su margen (0 si cae del lado correcto y fuera de la franja). $C\sum_i \xi_i$ penaliza esas violaciones en la función objetivo.
- $C \to \infty$: ninguna violación tolerada $\Rightarrow$ margen **duro** (A2). C finito $\Rightarrow$ margen **blando** (A3), con C controlando qué tan caro sale violarlo.

A4 · De clasificar a predecir un número

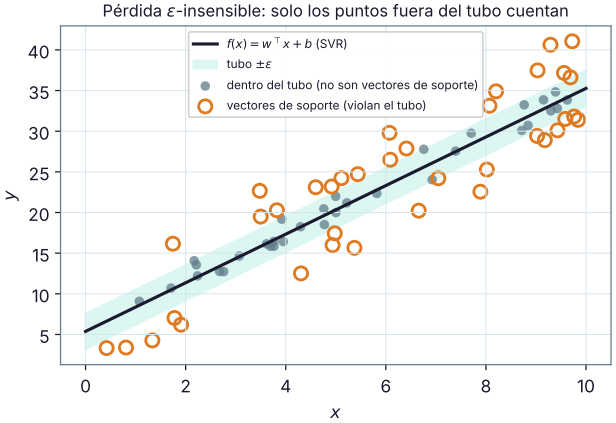
¿Cómo se traduce el margen a regresión?

En SVM (clasificación)		En SVR (regresión)
Frontera de decisión	$\rightarrow$	función de predicción $f(x)$
Margen (franja sin puntos)	$\rightarrow$	tubo de tolerancia $\pm\varepsilon$ alrededor de $f(x)$
Vectores de soporte: puntos **sobre** o **dentro** del margen	$\rightarrow$	vectores de soporte: puntos que **se salen** del tubo
C: penalización por violar el margen	$\rightarrow$	C: penalización por salirse del tubo

La pregunta de hoy

En SVM el margen separa dos clases. En SVR, ¿qué significa que una predicción "esté dentro del margen"? Eso es lo que viene en la parte B.

B1 · El tubo ε-insensible

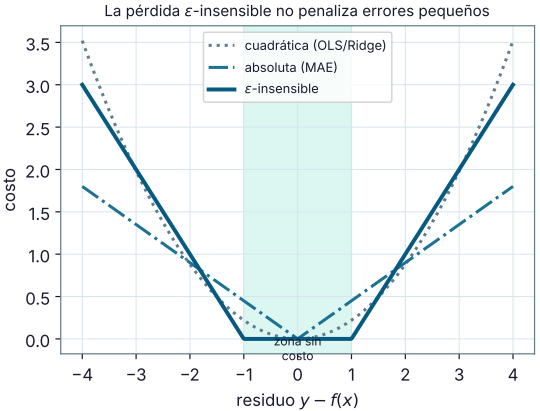


La idea central

- SVR dibuja una banda de ancho 2ε alrededor de $f(x)$: el **tubo**.
- Si una predicción cae **dentro** del tubo, se considera "suficientemente buena": **no se penaliza**, aunque no sea exacta.
- Solo los puntos que quedan **fuera** del tubo cuestan algo.

ε es una decisión del usuario: ¿qué tan grande es un error que no me importa? (Aquí, como en B3b, $f(x) = w^\top x + b$: una recta — el truco del kernel para curvarse viene hasta la parte C.)

B2 · La función de pérdida ε-insensible



Comparando pérdidas

- **Cuadrática** (OLS/Ridge): todo residuo cuesta, y los grandes cuestan **mucho más**.
- **Absoluta** (MAE): todo residuo cuesta, proporcional a su tamaño.
- **ε-insensible**: residuos menores a ε cuestan **cero**; más allá, crece de forma lineal.

Consecuencia

Al no exigir perfección, SVR ignora el ruido pequeño y se concentra en los errores que sí importan.

B3 · ¿Qué hace distinta la solución de SVR?

Dispersión (sparsity) en la solución

- En OLS o Ridge, **todas** las observaciones influyen en los coeficientes finales.
- En SVR, los puntos **dentro** del tubo no aportan nada a $f(x)$: la solución depende solo de los **vectores de soporte** (los que violan el tubo).
- Con pocos vectores de soporte, el modelo es más simple de evaluar y, a menudo, **más robusto** a ruido pequeño.

¿De qué depende cuántos vectores de soporte hay?

De dos hiperparámetros clave: C (parte A3) y ε (ancho del tubo). Vemos su efecto a continuación.

B3b · Formulación matemática de SVR

Problema primal (pérdida ε-insensible)

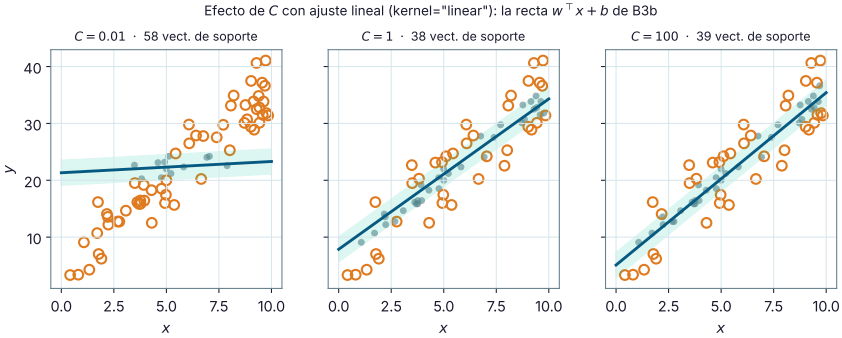
$$\min_{w,\,b,\,\xi,\,\xi^*} \quad \tfrac{1}{2}\|w\|^2 \;+\; C\sum_{i=1}^{n}\left(\xi_i + \xi_i^*\right)$$

$$\text{sujeto a} \quad \begin{cases} y_i - \left(w^\top x_i + b\right) \le \varepsilon + \xi_i \\ \left(w^\top x_i + b\right) - y_i \le \varepsilon + \xi_i^* \\ \xi_i,\ \xi_i^* \ge 0 \end{cases} \qquad i = 1, \ldots, n$$

Cómo leerlo

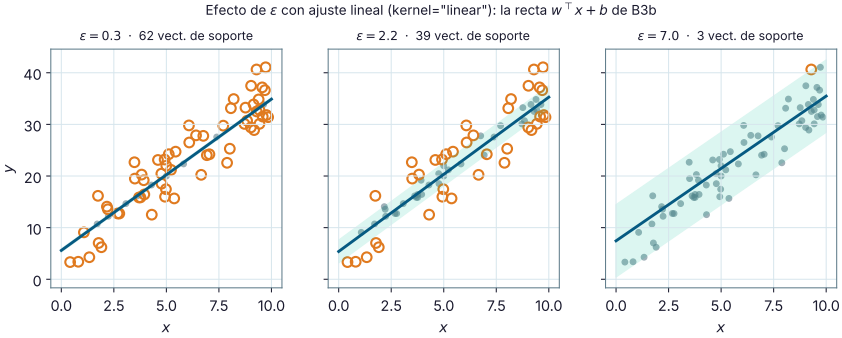
- Dos holguras por punto: ξ_i si la predicción queda **por debajo** del tubo, ξ_i^* si queda **por encima** (B1).
- Ambas son cero mientras el punto esté dentro de la banda $\pm\varepsilon$: por eso esos puntos no cuestan nada (B2) ni entran en la solución (B3).
- Misma estructura que SVM (A3b): mismo $\frac{1}{2}\|w\|^2$ y mismo papel de C — solo cambian las restricciones, porque ahora se penaliza salirse de un tubo y no estar del lado equivocado.

B4 · Efecto de C sobre el ajuste



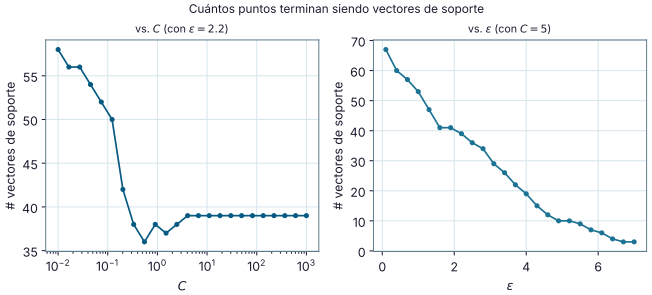
Con `kernel="linear"` el modelo **es** literalmente la recta $f(x) = w^\top x + b$ de B3b. C chica: tolera muchas violaciones y casi no sigue los datos (subajuste claro). C grande: persigue más los puntos, pero con un kernel lineal hay poco margen para "memorizar" — por eso el cambio entre $C = 1$ y $C = 100$ es modesto. Más adelante (C6), veremos qué tan distinto se ve esto con un kernel que sí puede curvarse.

B5 · Efecto de ε sobre el ajuste



Misma recta lineal de B3b. Con $\varepsilon = 0.3$ el tubo es angosto y 62 de 70 puntos quedan fuera de él (vectores de soporte); con $\varepsilon = 7.0$ el tubo es tan ancho que casi todos caben dentro y solo quedan 3. La recta en sí casi no cambia — lo que cambia es cuántos puntos **le cuestan algo** al modelo.

B6 · ¿Cuántos vectores de soporte terminamos usando?



Lectura

- A mayor C, **menos** vectores de soporte: el modelo se esfuerza más por ajustarse de cerca, así que menos puntos quedan fuera del tubo.
- A mayor ε, **menos** vectores de soporte: el tubo absorbe más puntos sin esfuerzo adicional.

C y ε se eligen juntos; nunca por separado.

C1 · El problema: relaciones que no son una línea

Un ejemplo cotidiano

El consumo eléctrico de un edificio **no** crece siempre igual con la temperatura exterior: baja hasta una temperatura de confort ($\approx 21^{\circ}$C) y vuelve a subir con el calor (aire acondicionado). Es una curva en **U**, no una recta.

¿Alcanza con SVR lineal?

Un SVR **lineal** solo puede dibujar una recta (o un hiperplano): nunca podrá seguir una curva en U, por más que ajustemos C o ε. Necesitamos que el modelo pueda **curvarse**.

Podríamos agregar x^2 como en la regresión polinomial... pero el **truco del kernel** logra el mismo efecto (y más) sin construir esas columnas a mano.

C2 · El truco del kernel: curvarse sin construir nuevas columnas

La idea

- SVR solo necesita comparar qué tan "parecidos" son dos puntos, a través de un **producto punto**.
- Una función **kernel** $K(x, x')$ calcula ese parecido **como si** los datos vivieran en un espacio de más dimensiones (donde sí serían lineales)...
- ...sin construir jamás esas dimensiones nuevas. Por eso es un "truco".

En la práctica

Solo se cambia el argumento `kernel` de `SVR`: `"linear"`, `"poly"` o `"rbf"`.

C2b · De la recta a $K(x, x')$: por qué basta un producto punto

Un hecho clave de la solución de B3b

El peso óptimo w siempre se puede escribir como una combinación de los propios puntos de entrenamiento:

$$w = \sum_{i=1}^{n} \beta_i \, x_i, \qquad \text{con } \beta_i = 0 \text{ en los puntos que } \textbf{no} \text{ son vectores de soporte (B3)}$$

Sustituyendo en $f(x) = w^\top x + b$

$$f(x) = \Big(\sum_{i=1}^{n} \beta_i \, x_i \Big)^\top x + b = \sum_{i=1}^{n} \beta_i \left(x_i^\top x \right) + b$$

Por qué importa

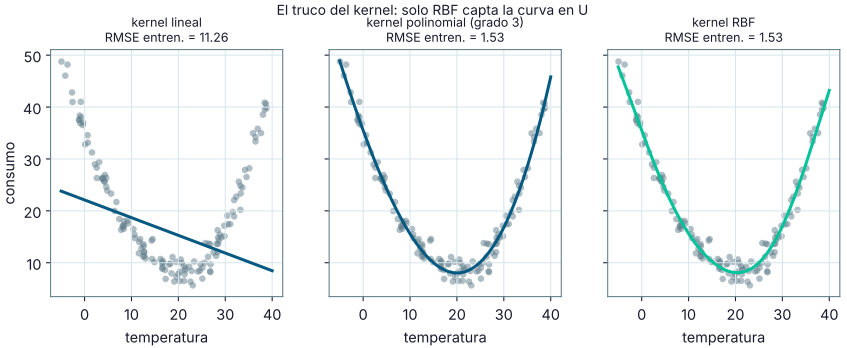
- $f(x)$ ya **no necesita** a w de forma explícita: solo usa el producto punto $x_i^\top x$ entre x y cada punto de entrenamiento.
- Por eso basta **reemplazar** ese producto punto por $K(x_i, x)$ (C1, C3): el modelo se curva, como si los datos vivieran en un espacio de más dimensiones, sin construir jamás ese espacio a mano.

C3 · Kernels comunes

Kernel	Fórmula	Cuándo usarlo
Lineal	$K(x, x') = x^\top x'$	la relación ya es (casi) lineal; rápido, pocos hiperparámetros
Polinomial	$K(x, x') = (\gamma\, x^\top x' + r)^d$	curvaturas suaves y conocidas (grado d)
RBF (gaussiano)	$K(x, x') = \exp(-\gamma \lVert x - x' \rVert^2)$	curvaturas arbitrarias; el más usado por defecto

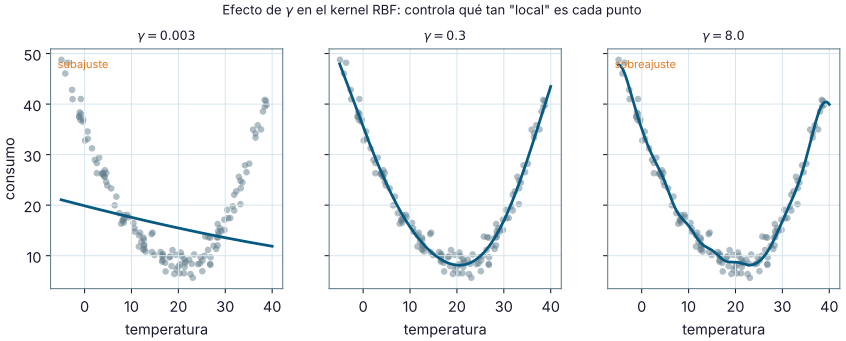
El parámetro γ (RBF, poly) controla qué tan "local" es la influencia de cada punto; d y r (poly) controlan el grado y el sesgo de la curva.

C4 · Comparación de los tres kernels en la misma curva



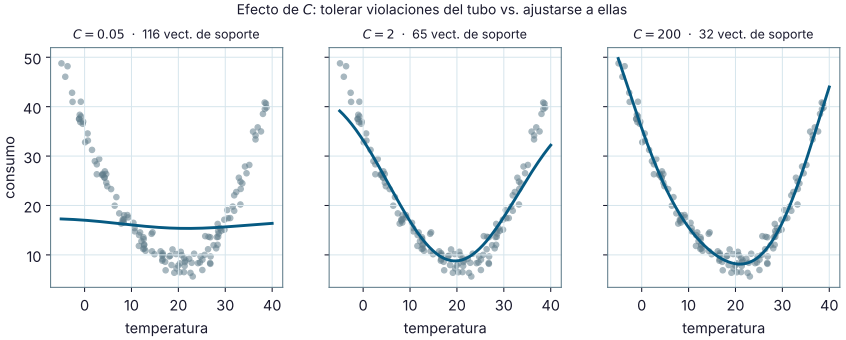
El kernel lineal no puede seguir la curva en U (alto error). Poly(3) y RBF sí la capturan, porque ambos permiten curvatura.

C5 · El parámetro γ en el kernel RBF



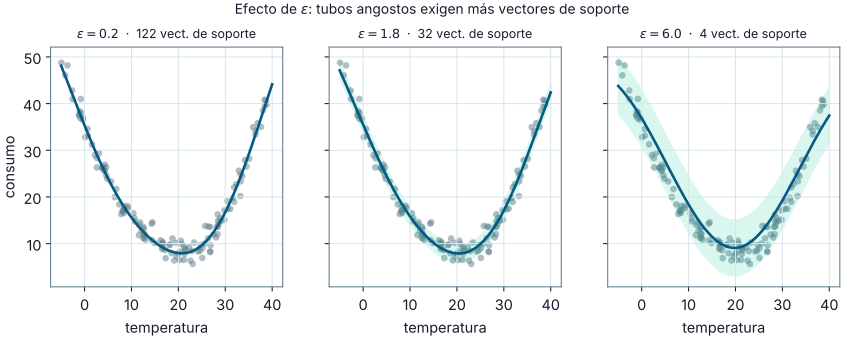
γ chico: cada punto influye en una región amplia (puede subajustar). γ grande: cada punto influye solo muy cerca de sí mismo (puede sobreajustar, como memorizar).

C6 · De vuelta a C, ahora con un kernel que sí se curva



En B4 vimos esto con `kernel="linear"`, donde C grande apenas cambiaba la recta. Aquí, con `kernel="rbf"` sobre los datos de consumo (curva en U), sí hay de sobra dónde "memorizar": C chica da un ajuste casi plano (subajuste); C grande persigue cada punto, incluido el ruido (sobreajuste). La diferencia entre ambas figuras es exactamente lo que gana (y arriesga) un kernel no lineal.

C7 · Y lo mismo con ε, con kernel RBF



Mismo patrón que B5, ahora con `kernel="rbf"`: ε chico exige seguir de cerca la curva $\rightarrow$ muchos vectores de soporte. ε grande deja que el tubo absorba buena parte de la curvatura $\rightarrow$ pocos vectores de soporte, ajuste más plano. En la parte D ajustamos C, ε **y** γ juntos, porque los tres interactúan.

D1 · Varios hiperparámetros que interactúan

El reto

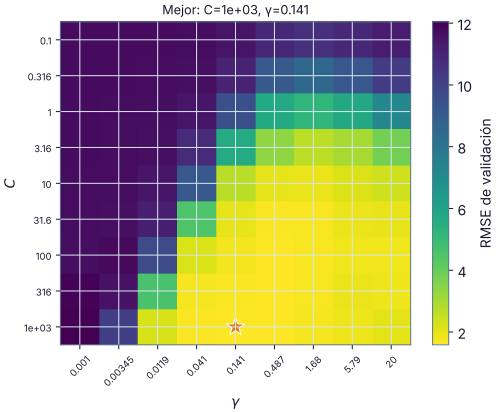
SVR con kernel RBF tiene (al menos) **tres** hiperparámetros: C, ε y γ. No se pueden elegir uno por uno: el mejor γ depende de C, y viceversa.

La solución: búsqueda en malla

`GridSearchCV` prueba **todas las combinaciones** de una rejilla de valores, evalúa cada una con **validación cruzada** y se queda con la de menor error.

Igual que en la Sesión 5 con `ElasticNetCV`/`GridSearchCV` para α y `l1_ratio`: la idea se repite, solo cambian los hiperparámetros.

D2 · `GridSearchCV` sobre C y γ



Lectura del mapa de calor

- Cada celda es el **RMSE de validación** promedio (5 particiones) para una combinación (C, γ).
- La estrella marca la **mejor** combinación encontrada.
- Zonas oscuras (arriba-izquierda): C chica y γ chica $\rightarrow$ modelo casi plano, subajuste.

ε también puede entrar a la malla; aquí se fijó para que el mapa quepa en 2D.

D3 · Buenas prácticas, igual que siempre

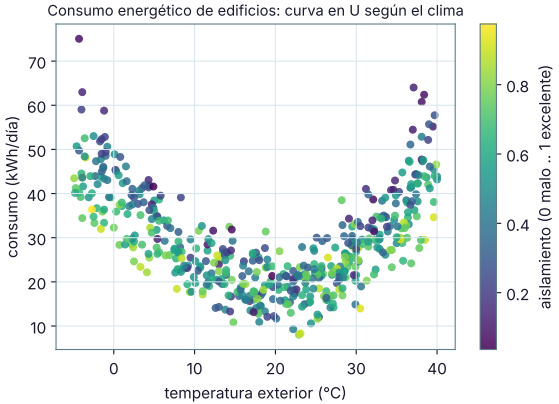
Tres hábitos que no cambian

- **Escalar siempre**: SVR con RBF usa distancias ($\|x - x'\|$); variables en escalas distintas dominan la distancia. `StandardScaler` va **dentro** del `Pipeline`.
- **Validación cruzada** para elegir C, ε, γ — nunca mirando el conjunto de prueba.
- El conjunto de **prueba** se evalúa **una sola vez**, al final, con los hiperparámetros ya elegidos.

Costo computacional

SVR puede ser lento en conjuntos grandes (n en decenas de miles o más): la búsqueda en malla crece rápido. Conviene acotar la rejilla con una búsqueda gruesa primero.

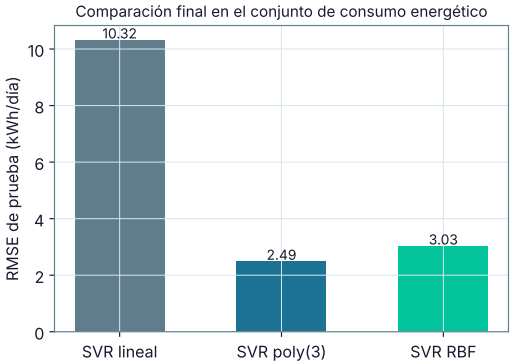
E1 · Los datos de hoy: consumo energético de edificios



El conjunto de datos

- Consumo diario (kWh) de edificios según **temperatura**, **superficie**, **ocupantes**, **aislamiento** y si es **fin de semana**.
- La curva en U se ve claramente: mínimo cerca de la temperatura de confort.
- Edificios mejor **aislados** (colores claros) sufren menos el clima: curva más plana.

E2 · Comparación final: lineal vs. polinomial vs. RBF



Lectura

- El kernel **lineal** queda muy por detrás: no puede seguir la curva en U.
- **Poly(3)** y **RBF** logran un RMSE parecido y mucho menor.
- Con más variables (no solo temperatura), RBF suele ser más flexible sin tener que adivinar el grado del polinomio.

Resumen de la sesión

Ideas clave

- **Margen blando**: tolerar algunas violaciones a cambio de un modelo más estable; C controla cuánto se tolera.
- **Tubo ε-insensible**: errores menores a ε no cuestan nada; solo los vectores de soporte (fuera del tubo) definen la solución.
- **Truco del kernel**: lineal, RBF y polinomial permiten curvarse sin construir columnas nuevas a mano.

Para llevarse

- C, ε y γ **interactúan**: se eligen juntos con `GridSearchCV` y validación cruzada.
- **Siempre** estandarizar dentro del `Pipeline`; SVR con RBF depende de distancias.
- Evaluar en prueba **una sola vez**, al final.

Siguiente sesión

Árboles de decisión para regresión y evaluación de modelos: un repaso de medio curso de todos los modelos de regresión vistos hasta ahora.

Actividad de laboratorio (individual)

SVR sobre un conjunto de datos de consumo de energía

Cuaderno de Jupyter con datos de consumo energético de edificios. Trabaja **de forma individual**.

1. Explora los datos y ajusta un **SVR lineal**: mide su RMSE de prueba.
2. Ajusta un **SVR con kernel RBF** con hiperparámetros por defecto y compáralo con el lineal.
3. Explora el efecto de C y de ε sobre el número de vectores de soporte y el error.
4. Compara los kernels **lineal**, **polinomial** y **RBF** con el mismo protocolo.
5. Usa `GridSearchCV` para elegir C, ε y γ del kernel RBF.
6. Evalúa el modelo final en el conjunto de **prueba** (una sola vez) y compáralo contra los modelos de regresión de sesiones anteriores.
7. Interpreta: ¿qué variables parecen importar más para el consumo? (apóyate en una gráfica de dependencia simple).

Entregable: el cuaderno con tus respuestas a preguntas de interpretación (¿por qué falla el kernel lineal aquí?, ¿qué combinación de C, ε, γ elegiste y por qué?).



¿Preguntas?

¡Pasemos a la actividad: SVR para consumo energético!
